

Pluggable OCR Architecture in Docling

Understanding how Docling decouples high-level visual document layout parsing from low-level optical character recognition backends.

Key Architectural Insight

Docling is not an OCR engine itself. It is a document intelligence and layout orchestration framework. Docling focuses on deep-learning structural models (e.g., layout segmentation via DocLayNet, reading order resolution, and TableFormer for complex tables), while delegating character recognition to external engines.

Pluggable OCR Architecture:

Docling provides abstract wrapper interfaces (`OcrEngine`) allowing you to swap OCR libraries based on performance, language support, or hardware:

DOCLING PROPRIETARY MODELS

- **DocLayNet Layout Model:** Bounding box classification for headers, body, pictures, and captions.
- **TableFormer:** Reconstructs complex hierarchical row/column spans and borderless grids.
- **Reading Order Heuristics:** Reconstructs human reading flow across multi-column pages.
- **Formula Enrichment:** LaTeX mathematical expression synthesis.

PLUGGABLE OCR BACKENDS

- **Character Recognition:** Pixel-level glyph decoding on scanned or degraded document regions.
- **Hardware Flexibility:** Swap between GPU-accelerated PyTorch, lightweight CPU ONNX, or Apple Silicon frameworks.
- **Multilingual Packs:** Leverage specialized language models (e.g., CJK, Indic, Arabic) per deployment.

SUPPORTED OCR BACKENDS COMPARISON

OCR BACKEND	KEY ARCHITECTURE & CHARACTERISTICS	RECOMMENDED USE CASE
EasyOCR Default	PyTorch-based end-to-end neural OCR. Strong multi-language support (80+ languages), native GPU acceleration support.	Default standard for general-purpose PDF/image parsing with GPU infrastructure available.
RapidOCR	Lightweight ONNX-runtime wrapper around PP-OCR models. Minimal dependencies, extremely high inference speeds on x86/ARM CPUs.	High-throughput batch processing and cloud serverless/microservices without dedicated GPUs.
Tesseract / Tesseractocr	Industry-standard mature open-source engine (LSTM-based). Supports extensive language dictionaries and custom trained data (<code>.traineddata</code>).	Enterprise on-premise Linux deployments requiring legacy language packs or specialized domain-trained models.
Mac OCR (ocrmac)	Native bridge to Apple's Vision Framework. Zero overhead, hardware-accelerated on Apple Silicon (M1/M2/M3/M4/M5).	Local development, Mac desktop tooling, and edge processing on Apple hardware.

PYTHON IMPLEMENTATION EXAMPLE

```
from docling.datamodel.base_models import InputFormat
from docling.datamodel.pipeline_options import (
    PdfPipelineOptions,
    EasyOcrOptions,
    TesseractOcrOptions,
    RapidOcrOptions,
)
from docling.document_converter import DocumentConverter, PdfFormatOption

# 1. Configure the pipeline and enable OCR
pipeline_options = PdfPipelineOptions()
pipeline_options.do_ocr = True

# 2. Select and customize the OCR Backend
# Option A: RapidOCR for ultra-fast CPU batch inference
pipeline_options.ocr_options = RapidOcrOptions()

# Option B: Tesseract with custom multilingual packs
# pipeline_options.ocr_options = TesseractOcrOptions(lang=["eng", "deu"])

# Option C: EasyOCR with GPU acceleration
# pipeline_options.ocr_options = EasyOcrOptions(lang=["en"], use_gpu=True)

# 3. Instantiate converter with tailored configuration
converter = DocumentConverter(
    format_options={
        InputFormat.PDF: PdfFormatOption(pipeline_options=pipeline_options)
    }
)

# 4. Execute conversion
result = converter.convert("scanned_document.pdf")
doc = result.document
print(doc.export_to_markdown())
```